Finally (though not always), the OS talks to the world—it provides a user interface, which lets users give instructions to the OS or run applications of their choice. Earlier, OSes used the *command line interface*, where users would type in what they wanted the OS to do. Today's OSes give us a *Graphical User Interface (GUI)*, pronounced "gooey". We use the mouse to point to and click on the pretty icons and they magically start our programs.

While the things they do remain the same all over, different operating systems do things differently, depending on what they are designed for.

1.1.1 Real-time Operating Systems (RTOS)

Real-time operating systems are no-frills operating systems designed for one thing—performance. They are typically used in scientific research, industrial robots and devices like mobile phones. They offer little, if any, user interaction.

Real-time computing means that the system must conform to very strict time-constraints or deadlines. For example, if the time taken for a robot to lift an object and move it from point A to point B is 10 seconds, then it must be 10 seconds always—nothing less, nothing more. Imagine what would happen if a car's fuel injection system injected petrol into the engine before it was ready for it—at best, the engine might sputter, but at worst, it could suffer serious damage!

Real-time OSes are the toughest to write—programmers need to use specialised algorithms to manage resources and ensure that the system's deadlines are always met.

1.1.2 Single-Task Operating Systems

As the name suggests, these operating systems are capable of running only one task at a time. All system resources are devoted to this single task, and are returned to the OS once the task is complete. Most mobile phone OSes are single-task systems.